

Answer the questions in the boxes provided on the question sheets. If you run out of room for an answer, add a page to the end of the document.

Related Readings:

- Proof Primer: <https://canvas.wisc.edu/courses/485480/files/50373320>
- CS240 Readings: <http://pages.cs.wisc.edu/~hasti/cs240/readings/>

Name: Xu Ming Huang

Wisc id: 908 772 9324

Induction

1. Prove the following by induction.

(a) $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Solution:

$$\text{Let } F(n) = \sum_{i=1}^n i, G(n) = \frac{n(n+1)}{2}$$

$$\text{Base Case: } F(1) = \sum_{i=1}^1 i = 1 = \frac{1 \times (1+1)}{2} = G(1)$$

$$\text{Let } F(k) = \sum_{i=1}^k i = \frac{k(k+1)}{2} = G(k)$$

$$F(k+1) = \sum_{i=1}^{k+1} i = \sum_{i=1}^k i + k+1 = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2} = G(k+1)$$

$$\text{Thus } \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

(b) $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

Solution:

$$\text{Let } F(n) = \sum_{i=1}^n i^2, G(n) = \frac{n(n+1)(2n+1)}{6}$$

$$\text{Base: } F(1) = \sum_{i=1}^1 i^2 = 1 = \frac{1 \times (1+1) \times (2 \times 1 + 1)}{6} = G(1)$$

$$\text{Let } F(k) = \sum_{i=1}^k i^2 = \frac{k(k+1)(2k+1)}{6} = G(k)$$

$$\text{then } F(k+1) = \sum_{i=1}^{k+1} i^2 = \sum_{i=1}^k i^2 + (k+1)^2 = \frac{1}{6} k(k+1)(2k+1) + (k+1)^2$$

$$\Rightarrow = \left[\frac{1}{6} k(2k+1) + k+1 \right] (k+1) = \frac{1}{6} (k+1) (2k^2 + 7k + 6) = \frac{1}{6} (k+1) (k+2) (2k+3) = G(k+1)$$

$$\text{Thus } \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

(c) $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$

Solution:

Let $F(n) = \sum_{i=1}^n i^3$, $G(n) = \frac{n^2(n+1)^2}{4}$

Base: $F(1) = \sum_{i=1}^1 i^3 = 1 = \frac{1^2(1+1)^2}{4} = G(1)$

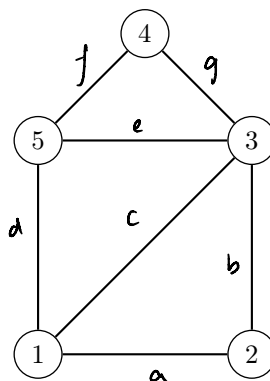
Assume $F(k) = \sum_{i=1}^k i^3 = \frac{k^2(k+1)^2}{4} = G(k)$

Then $F(k+1) = \sum_{i=1}^{k+1} i^3 = \sum_{i=1}^k i^3 + (k+1)^3 = \frac{k^2(k+1)^2}{4} + (k+1)^3 = \frac{1}{4}(k+1)^2(k^2 + 4k + 4) = \frac{1}{4}(k+1)^2(k+2)^2$

Thus $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$

Graphs and Trees

2. Give the adjacency matrix, adjacency list, edge list, and incidence matrix for the following graph.



Solution:

Adjacency Matrix

	1	2	3	4	5
1	0	1	1	0	1
2	1	0	1	0	0
3	1	1	0	1	1
4	0	0	1	0	1
5	1	0	1	1	0

Adjacency List

Array

1 → 2 → 3 → 5
 2 → 1 → 3
 3 → 1 → 2 → 3 → 4 → 5
 4 → 3 → 5
 5 → 1 → 3 → 4

Edge List = [

[1, 2],
 [1, 3],
 [1, 5],
 [2, 3],
 [3, 4],
 [3, 5],
 [4, 5],
]

Incidence Matrix

	a	b	c	d	e	f	g
1	1	0	1	1	0	0	0
2	1	1	0	0	0	0	0
3	0	1	1	0	1	0	1
4	0	0	0	0	0	1	1
5	0	0	0	1	1	1	0

3. How many edges are there in a complete graph of n vertices? Prove by induction.

Solution:

Assume $F(n) = n(n-1)/2$

Base $F(1) = 1 \times 0 / 2 = 0$

Assume $F(k) = k(k-1)/2$

When add one more vertex to $F(k)$, all k vertex should attach an edge to it

then $F(k+1) = k + F(k) = \frac{1}{2}(k-1)k + k = \frac{1}{2}k(k-1+2) = \frac{1}{2}k(k+1)$

which is equal to RHS

Proofs

6. Show that $2x$ is even for all $x \in \mathbb{N}$.

(a) By direct proof.

Solution:

$$\text{Let } x = \begin{cases} 2i+1, & \text{odd} \\ 2i, & \text{even} \end{cases}$$

$$\text{then } 2x = \begin{cases} 4i+2 = 2(2i+1), & \text{even} \\ 4i = 2 \cdot 2i, & \text{even} \end{cases}$$

(b) By contradiction.

Solution:

Assume $2x$ is odd for all $x \in \mathbb{N}$

when $x = 0 \in \mathbb{N}$, $2x = 0$ which is even

so contradiction.

7. For all $x, y \in \mathbb{R}$, show that $|x + y| \leq |x| + |y|$. (Hint: use proof by cases.)

Solution:

if $x, y \leq 0$ or $x, y \geq 0$,

$$|x+y| = |x| + |y|$$

else if $x \geq 0, y \leq 0$ or $x \leq 0, y \geq 0$

$$|x+y| < |x| + |y|$$

overall, $|x+y| \leq |x| + |y|$

Program Correctness (and Invariants)

8. For the following algorithms, describe the loop invariant(s) and prove that they are sound and complete.

Algorithm 1: findMin

(a) **Input:** a : A non-empty array of integers (index starting at 1)
Output: The smallest element in the array

```

begin
   $min \leftarrow \infty$ 
  for  $i \leftarrow 1$  to  $\text{len}(a)$  do
    if  $a[i] < min$  then
       $min \leftarrow a[i]$ 
    end
  end
  return  $min$ 
end
```

Solution:

Loop Invariant:

At the start of iteration i , min is the smallest element in $A[0 \dots i-1]$

Initialization:

$min = \infty$ before $i=1$, which is true because no elements of A have been examined yet

Maintenance:

Assume at the start of iteration i , $min = \min(A[0], \dots, A[i-1])$

then if $A[i] < min$, then $min = A[i]$, thus min remains the smallest element

else $A[i] > min$, min remains and still be the smallest.

Thus, at the start of next iteration $i+1$, the invariant holds again.

Termination:

the loop ends when $i = \text{len}(A)$

By the invariant $min = \min(A[0] \dots A[\text{len}(A)])$

Therefore, $min = \min(A)$

So findmin is correct.

Algorithm 2: InsertionSort

Input: a : A non-empty array of integers (index starting at 1)
Output: array ' a ' sorted from largest to smallest

```

begin
  for  $i \leftarrow 2$  to  $\text{len}(a)$  do
     $val \leftarrow a[i]$ 
    for  $j \leftarrow 1$  to  $i - 1$  do
      if  $val > a[j]$  then
        shift  $a[j..i - 1]$  to  $a[j + 1..i]$ 
         $a[j] \leftarrow val$ 
        break
      end
    end
  end
  return  $a$ 
end
  
```

(b)

Solution:

At start of iteration i , the invariant is that $a[1]$ to $a[i]$ is sorted from largest to smallest

Initialization:

Before first iteration $i=2$, $[a[1], a[i-1]]$ is just $a[1]$ which is abs sorted. So the invariant holds.

Maintenance:

Assume at iteration $i-1$, assume $[a[1], a[i-1]]$ is sorted. Then when it goes to i

If $val = a[i] > a[j]$, $j \in [1, i-1]$, then insert $a[i]$ between $a[j]$ and $a[j-1]$, thus $[a[1], a[i]]$ is still sorted
 else $val = a[i] < a[j]$, $j \in [1, i-1]$, then automatically $a[i]$ stay at the end of $[a[1], a[i]]$ because it's smallest, thus $[a[1], a[i]]$ remains sorted

Therefore, invariant holds.

Termination:

When iteration comes to end, $i = \text{len}(a)$, $[a[1], a[n]]$ is sorted. Thus, it's correct.

Coding Question: Hello World

Most assignments will have a coding question. You can code in C, C++, C#, Java, Python, or Rust. You will submit a Makefile and a source code file.

Makefile: In the Makefile, there needs to be a build command and a run command. Below is a sample Makefile for a C++ program. You will find this Makefile in assignment details. Download the sample Makefile and edit it for your chosen programming language and code.

```
#Build commands to copy:
#Replace g++ -o HelloWorld HelloWorld.cpp below with the appropriate command.
#Java:
#    javac source_file.java
#Python:
#    echo "Nothing to compile."
#C#:
#    mcs -out:exec_name source_file.cs
#C:
#    gcc -o exec_name source_file.c
#C++:
#    g++ -o exec_name source_file.cpp
#Rust:
#    rustc source_file.rs

build:
    g++ -o HelloWorld HelloWorld.cpp

#Run commands to copy:
#Replace ./HelloWorld below with the appropriate command.
#Java:
#    java source_file
#Python 3:
#    python3 source_file.py
#C#:
#    mono exec_name
#C/C++:
#    ./exec_name
#Rust:
#    ./source_file

run:
    ./HelloWorld
```

9. HelloWorld Program Details

The input will start with a positive integer, giving the number of instances that follow. For each instance, there will be a string. For each string s , the program should output Hello, s ! on its own line.

A sample input is the following:

```
3
World
Marc
Owen
```

The output for the sample input should be the following:

```
Hello, World!
Hello, Marc!
Hello, Owen!
```